

to know if it is working. The advantage of the *subprocess* command is that you can start *ping* on each device without waiting for it to finish. You can use threads without the need to manage the complexity.

```
from subprocess import Popen, PIPE
# The addresses of localhost, the wireless router, the modem, the main
desktop
hosts = ['127.0.0.1', '192.168.0.1', '192.168.1.1', '192.168.0.100']
procs = []
for host in hosts:
    procs.append(Popen(['ping', '-c2', '-f', host], stdout=PIPE))
for proc in procs:
    proc.wait()
for proc in procs:
    print proc.communicate()[0]
```

Create a list of devices/hosts in which you are interested. You then start a *ping* process for each of the hosts, with parameters limiting the number of times you *ping* while reporting only the summary information. Next, you wait for each process to get over and then process the results. If successful, you will see an output like the following:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
--- 127.0.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1008ms
```

```
rtt min/avg/max/mdev = 0.063/0.102/0.122/0.021 ms
```

While for an unsuccessful case...

```
PING 192.168.0.100 (192.168.0.100) 56(84) bytes of data.
--- 192.168.0.100 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 3001ms
```

The nice thing about scripts is that you can keep adding them to your armoury as and when you come across problems. For example, even though the disk capacities are very large, a partition may still run out of space. It can be very hard to figure out why some applications or even the GUI desktop just refuses to start. Unresponsive or erratic DNS servers can create a frustrating browsing experience.

Scripts that monitor the systems can help you prevent problems before they arise, while sitting anywhere in the world with just a network connection. The downside is that your relatives may not realise your contribution or utility! This can be a serious issue in the corporate environment, but you can always write scripts so that your presence is indispensable—for example, by converting your scripts to one of the write-only scripting languages! 

By: Dr Anil Seth

The author is a consultant by profession and can be reached at seth.anil@gmail.com



Calling...

Share your knowledge

Inviting FOSS experts to write articles on their area of interest

LFY covers a myriad of topics—network management, software development, embedded systems, community issues, and even hands-on guide for newbies. If you've got an interesting topic, let us know. Thanks to the launch of www.openTis.com (aka linuxforu.com) we are now trying to extend our content portfolio related to Linux & Open Source.

To know more on how to become an LFY author, contact us at

lfyedit@efyindia.com

Few topics that top our list:

- ◆ Tips 'n' Tricks for software developets or IT implementers
- ◆ Cool tweaks for FOSS enthusiasts
- ◆ FOSS on mobile
- ◆ Virtualisation (Implementation)
- ◆ OpenJDK or Java on Linux
- ◆ OpenSolaris (software development)
- ◆ How can I do 'that' on Linux
- ◆ Reviews of latest open source projects & tools

